IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2019

EEE/EIE PART II: MEng, BEng and ACGI

Corrected Copy

ALGORITHMS AND COMPLEXITY

Tuesday, 28 May 10:00 am

Time allowed: 1:30 hours

There are TWO questions on this paper.

Answer ALL questions. Question One carries 40% of the marks. Question Two carries 60%.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : D.F.M. Goodman
Second Marker(s) : D.B. Thomas

© Imperial College London

ALGORITHMS AND COMPLEXITY

1. a) Give a tight bound for the running time of each of the following algorithms as a function of the variable n, and justify your result. You may assume basic arithmetic operations and memory allocation are $O(1)$.

i) Compute the outer product $\mathbf{x}\mathbf{y}^\intercal$ of two column vectors $\mathbf{x}$, $\mathbf{y}$ of length n.

```
function OUTERPRODUCT(x, y, n)
    Allocate memory for an n × n matrix C.
    for i = 0 to n − 1 do
        for j = 0 to n − 1 do
            C_ij = x_i y_j
        end for
    end for
    return C
end function
```

[5]

ii) Given an item x and a sorted array A of length n, compute whether or not the item x is in A using bisection:

```
function ELEMENTINSORTEDARRAY(x, A)
    L = 0
    R = len(A) − 1
    while L ≤ R do
        m = floor((L+R)/2)
        if A_m = x then
            return True
        else if A_m < x then
            L = m+1
        else
            R = m−1
        end if
    end while
    return False
end function
```

[5]

b) Give a tight bound for each of the following recurrence relations. Note that the Master Theorem is stated in full on the final page.

i) $T(n) = 4T(n/5) + O(\sqrt{n})$. [2]

ii) $T(n) = \max\{1, nT(n-1)\}$. [2]

iii) $T(n) = 5T(n/25) + \sqrt{n} + \sqrt[3]{n}$. [2]

iv) $T(n) = 32T(n/2) + O(n^6)$. [2]

v) $T(n) = (16T(3n/4) + n^{1.5})/9$. [2]

2. The merge sort algorithm is a divide and conquer algorithm for sorting an array of n items. It works by:

- **Dividing** the array into two sub-arrays of approximately equal size.
- **Sorting** the sub-arrays recursively.
- **Combining** the sorted sub-arrays into a single sorted array by jointly iterating over them.

The following pseudocode implements this algorithm:

```
1: function MERGESORT(x)
2:     n = len(x)
3:     if n ≤ 1 then
4:         return x
5:     end if
6:     m = floor(n/2)
7:     L = MERGESORT(subarray of items 0 to m − 1 of x)
8:     R = MERGESORT(subarray of items m to n − 1 of x)
9:     y = array of size n
10:    i = 0, j = 0
11:    for k = 0 to n − 1 do
12:        r = False
13:        if i = m then
14:            r = True
15:        else if j < n − m then
16:            r = (R_j ≤ L_i)
17:        end if
18:        if r then
19:            y_k = R_j
20:            j = j + 1
21:        else
22:            y_k = L_i
23:            i = i + 1
24:        end if
25:    end for
26:    return y
27: end function
```

a) Explain the merging step (lines 11-25). [3]

b) Calculate a tight upper bound on the complexity of the algorithm. [3]

c) The *ordering* of a sorting algorithm's output is the output order of items from the input array. Assume each item is unique (no repeated items). Every possible ordering (or permutation) of n items must be a possible outcome of a sorting algorithm.

i) The array $\{A, B, C\}$ has 6 possible orderings after sorting: *ABC*, *ACB*, *BAC*, *BCA*, *CAB*, *CBA*. What is the number of possible outcomes when sorting an array of n unique items? [2]

ii) If we know the result of a single comparison between two items in an array, this eliminates some possible orderings. Given knowledge of any single comparison, how many remaining outcomes are there for the sorted array? [2]

iii) In the worst case, how many comparisons between items in an array would be required to fully sort the array? [3]

iv) Using Ω notation, find a lower complexity bound for the number of operations for a sorting algorithm that can only use comparisons. Express your answer using only combinations of logarithmic, polynomial or exponential functions of n. [4]

d) Consider an array x of $n = 2^m$ items. Consider the recursive depth d of each function call in the execution of the merge sort algorithm. So, the first function call executed is at depth $d = 1$ and this is the only function call executed at this depth. This first function call recursively calls merge sort twice (for the arrays L and R), and these two function calls will be at depth $d = 2$, and so on.

At depth d, each function call is required to combine two sorted arrays, each of size 2^{m-d}, into a single sorted array of size 2^{m-d+1}. There are 2^{d-1} such function calls executed.

Let C_d refer to the total number of comparisons $R_j \leq L_i$ made on line 16 across all function calls executed at depth d.

i) What is the minimum possible number of comparisons C_d at depth d?

Hint: first consider how many comparisons are made in a single function call at this depth. [4]

ii) What is the maximum possible number of comparisons C_d at depth d? [4]

iii) Give an exact upper and lower bound for the total number of comparisons $C = \sum_{d=1}^{m} C_d$ required by the merge sort algorithm for $n = 2^m$ items. [3]

iv) Find an array x of $n = 4$ items that takes the maximum number of comparisons using merge sort, and an array of the same size that takes the minimum number of comparisons. [2]

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) + O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$